

LLM4JMH: Studying the Use of LLMs for Generating Java Performance Microbenchmarks

Zongxiong Chen
Fraunhofer FOKUS
Berlin, Germany
zongxiong.chen@fokus.fraunhofer.de

Derui Zhu*
Rochester Institute of Technology
Rochester, USA
Technical University of Munich
Munich, Germany
derui.zhu@tum.de

Kundi Yao
University of Waterloo
Waterloo, Canada
kundi.yao@uwaterloo.ca

Weiyi Shang
University of Waterloo
Waterloo, Canada
wshang@uwaterloo.ca

Jinfu Chen
Wuhan University
Wuhan, China
jinfuchen@whu.edu.cn

Jiahui Geng
Linköping University
Linköping, Sweden
jiahui.geng@mbzuai.ac.cn

Alexander Pretschner
Technical University of Munich &
Fortiss
Munich, Germany
alexander.pretschner@tum.de

Jens Grossklags
Technical University of Munich
Munich, Germany
jens.grossklags@in.tum.de

Manfred Hauswirth
Fraunhofer FOKUS
Berlin, Germany
Technische Universität Berlin
Berlin, Germany
manfred.hauswirth@fokus.fraunhofer.de

Sonja Schimmler
Fraunhofer FOKUS
Berlin, Germany
Technische Universität Berlin
Berlin, Germany
sonja.schimmler@fokus.fraunhofer.de

Abstract

Performance regressions pose a significant threat to software reliability. In recent years, smaller-scale performance testing, such as performance microbenchmarks, has been adopted in practice to detect such regressions in an early stage of development. Developing these microbenchmarks is costly, error-prone, and demands specialized expertise. Motivated by the recent progress of large language models (LLMs) in code-related tasks, we study whether LLMs can step in as performance experts to automate the generation of microbenchmarks for performance regression detection. While existing approaches typically depend on functional unit tests to generate performance microbenchmarks, we instead design an LLM-based approach, *LLM4JMH*, to assess the capability of LLMs in generating reliable and effective performance microbenchmarks directly from source code. We further explore how program analysis techniques, such as static analysis, can enhance the reliability and effectiveness of LLM-generated performance microbenchmarks. Experimental

results show that LLM-generated performance microbenchmarks achieve comparable bug detection effectiveness to expert-written microbenchmarks, while reducing overall expected detection latency by up to 52.60% across *RxJava*, *Eclipse Collections*, and *Zipkin*. In the study, the generated tests can successfully identify six out of 11 real-world performance bugs in *Apache Flink*. These results demonstrate that LLM-based performance microbenchmark generation can automate early performance regression detection in continuous integration pipelines of the software development process and reduce reliance on expert-crafted tests, advancing performance-aware software engineering.

CCS Concepts

• **Software and its engineering** → **Software maintenance tools.**

Keywords

Microbenchmark Generation, Large Language Models, Performance Regression Detection, Software Testing Automation

ACM Reference Format:

Zongxiong Chen, Derui Zhu, Kundi Yao, Weiyi Shang, Jinfu Chen, Jiahui Geng, Alexander Pretschner, Jens Grossklags, Manfred Hauswirth, and Sonja Schimmler. 2026. LLM4JMH: Studying the Use of LLMs for Generating Java Performance Microbenchmarks. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26)*, April 12–18, 2026, Rio de Janeiro,

*Corresponding author.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

ICSE '26, Rio de Janeiro, Brazil

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2025-3/26/04

<https://doi.org/10.1145/3744916.3787777>

Brazil. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3744916.3787777>

1 Introduction

Performance regression is a common but often overlooked problem in modern software systems [17, 19, 57]. Even small performance regressions can degrade user experience or increase costs, yet these issues are rarely detected by standard functional tests [48]. Performance microbenchmarks are one of the most effective ways to catch performance regressions through testing [23, 44], and have become standard in areas such as operating systems [45], virtual machines [7], and high-performance computing [35]. However, writing performance microbenchmarks is time-consuming and requires substantial manual effort and deep domain expertise [7, 8, 16]. Moreover, the lack of robust tooling deepens these challenges, hindering the widespread adoption of performance microbenchmark test practices [8, 20]. Therefore, most projects lack sufficient performance tests, and performance regressions are often shipped into production unnoticed.

To alleviate the substantial manual effort involved in crafting performance microbenchmarks, several automated approaches have been proposed to generate microbenchmarks compatible with the Java Microbenchmark Harness (JMH) [37]. However, these methods exhibit critical limitations that hinder their practical effectiveness. For instance, AutoJMH [43] often generates overly simplistic microbenchmarks that fail to reflect realistic workload patterns or sufficiently explore complex code paths, thereby limiting their utility in detecting meaningful performance regressions. A more recent approach, *ju2jmh* [20], attempts to convert existing JUnit functional test cases into JMH-compatible performance microbenchmarks. Although *ju2jmh* leverages existing test suites to maintain test coverage, it suffers from redundant microbenchmark generation, insufficient workload generation, and a tendency to produce unstable microbenchmarks. Critically, the microbenchmarks produced by such automated methods still fall short of the reliability and detectability offered by well-crafted manual microbenchmarks.

Motivated by the recent progress of large language models (LLMs) in generating source code and test cases across various software engineering tasks [41, 58], we conduct an empirical study to answer the question: *Is an LLM capable of generating effective microbenchmarks to detect performance regressions?* To answer this question, we present the first empirical study of using LLMs to generate JMH microbenchmarks in Java applications. Specifically, we design an approach, *LLM4JMH*, for (1) assessing whether LLMs can produce stable and reliable test cases for performance microbenchmarks, (2) detecting injected and real-world performance regressions, (3) comparing their effectiveness with expert-written and existing automated baselines. To further demonstrate the potential of LLM-assisted performance microbenchmark generation, we incorporate program analysis techniques (e.g., static analysis, call graphs) to improve the quality of our approach.

To evaluate the effectiveness of *LLM4JMH*, we conduct experiments on three widely used Java projects (*RxJava*, *Eclipse Collections*, and *Zipkin*), where performance regressions are injected using mutation testing with heavy-weight operators. Additionally, we assess *LLM4JMH* on *Apache Flink*, which targets real-world performance

regressions that were reported and addressed by developers. The empirical results demonstrate that microbenchmarks generated by *LLM4JMH* outperform expert-written JMH test suites in detecting performance regressions. Furthermore, we validate the effectiveness of *LLM4JMH* in identifying real-world performance regressions, successfully uncovering six of 11 issues in *Apache Flink*.

In summary, this paper makes the following contributions:

- To the best of our knowledge, we conduct the first empirical study to explore the effectiveness of LLMs in generating performance microbenchmarks to detect performance regression in the early software development stage (continuous integration process).
- We conduct a comprehensive experiment to study the feasibility of applying LLMs to generate a performance microbenchmark for uncovering known and unknown performance regressions.
- We demonstrate the practical applicability of using LLMs for early-stage performance regression detection, showing their effectiveness in real-world software systems, lowering the barrier to systematic automated performance regression detection.

2 Methodology

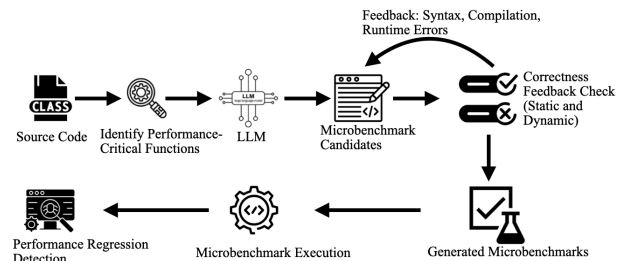


Figure 1. Overview of *LLM4JMH*: The diagram illustrates an LLM-driven pipeline that generates performance microbenchmarks, using feedback on syntax, compilation, and runtime errors to iteratively refine and produce valid benchmarks.

To systematically investigate the effectiveness of LLMs to generate performance microbenchmarks, we conduct an empirical study centered on JMH test suite generation and evaluation. Figure 1 outlines the overall design of *LLM4JMH*, which addresses three key challenges: the high cost and expertise required for performance microbenchmark design, the scalability of large-scale microbenchmark generation, and the instability of poorly generated microbenchmarks. Specifically, *LLM4JMH* consists of the following steps. First, we employ LLMs to generate candidate microbenchmarks directly from the codebase on a file-by-file basis. Since source files are independent, generation can proceed in parallel, making scalability depend only on the LLMs' context window rather than the repository size. Next, we validate the generated candidates through automated checks at the syntax, compilation, and runtime levels. Finally, we systematically assess the validated microbenchmarks on several widely used Java projects to assess their effectiveness in detecting both injected and real-world performance regressions, as well as their measurement stability. We further compare the results against expert-written and automated baseline benchmarks. Each of these stages is described in detail in the following subsections.

2.1 Microbenchmark Candidate Generation

To generate candidate microbenchmarks, we begin by providing the LLM with a carefully designed prompt. The prompt design followed an iterative, human-in-the-loop process with LLM assistance. We first defined the objective, i.e. generating correct, runnable JMH microbenchmarks for a given source code snippet following best practices, and asked the LLM to propose multiple prompt variants. Each was tested on a small set of representative classes, with JMH experts assessing the resulting benchmark quality. Based on these trials, we selected and refined the best-performing prompt through iterative LLM-guided adjustments and expert review until the generated microbenchmarks were consistently sound. This prompt directs the model to examine a given code unit and decide whether it is suitable for performance benchmarking leveraging the internal knowledge from the LLM. If the code is not appropriate for benchmarking, the model is instructed to return a simple *SKIP* keyword along with a short explanation. If the code can be benchmarked, the model generates a valid JMH microbenchmark in throughput mode, formatted using markdown-style code blocks to facilitate automatic extraction. Once we obtain the generated microbenchmarks, we apply a structured post-processing step to remove any irrelevant text, ensure that class and file names are consistent, eliminate extraneous explanations, and fix missing import statements where possible. Nevertheless, the generated microbenchmarks may still suffer from undeclared variables, invalid method calls, or incorrect API usage, as the LLMs do not have a solid understanding of the given project's context. Consequently, we treat the generated microbenchmark as a plausible candidate, which is then passed to the next stage for further validation of syntax, compilation, and runtime behavior.

2.2 Correctness Feedback Check

Although LLMs can generate seemingly valid microbenchmarks, correctness is not guaranteed. To ensure that only syntactically and semantically valid microbenchmarks are selected, we embed generation in a structured correctness-feedback loop. Each candidate undergoes a sequence of validation steps:

- **Syntax Checker** validates that the generated microbenchmark adheres to Java syntax rules, includes necessary import statements, and correctly applies JMH-specific annotations and idioms to be parsed and understood by downstream tools.
- **Compilation Checker** attempts to compile the generated benchmark alongside the target system and its dependencies. This step detects issues such as incorrect method signatures, invalid API usage, missing types, or dependency mismatches that may result from incomplete or outdated model knowledge.
- **Runtime Checker** executes the compiled benchmark in a controlled (sandboxed) environment to confirm a successful completion. This ensures the benchmark targets the intended method, exercises meaningful code paths, and avoids common runtime failures such as *NullPointerException* or *OutOfMemoryError*.

A key feature of this design is a bounded retry mechanism: At each stage, the LLM is tuned to T attempts (i.e., $T = 10$ empirically for optimal generalizability and cost-efficiency (see Figure 7b)) to repair invalid output. Diagnostic feedback (e.g., syntax errors,

Table 1. Overview of the studied subjects. Contr. refers to the number of contributors. SLOC is the source lines of code.

Subject	Version	Commit SHA	Stars	Contr.	SLOC
RxJava	3.28.16	67e949db	48.1k	299	315,482
Eclipse Collections	12.0.0	b89e089b	2.5k	115	317,975
Zipkin	2.24.0	a36e0577	17.2k	156	39,915

compiler stack traces, or runtime exceptions) is converted into structured, human-readable prompts that guide the LLM in refining its next attempt. If a microbenchmark fails the same check more than T times, the pipeline conservatively discards generation for that method. This prevents infinite loops and focuses computational resources on feasible, high-quality tests.

This correctness feedback check serves three purposes: (1) It filters out candidates that are impossible to fix due to ambiguous or highly coupled source logic. (2) It gives the LLM multiple opportunities to resolve common generation pitfalls, such as missing imports, misapplied annotations, or data mismatches. (3) It ensures that the generated benchmarks exercise meaningful code paths, where we define a microbenchmark as meaningful only if it is compilable and executable. Microbenchmarks that fail to compile or execute are considered non-meaningful, since they cannot exercise the targeted benchmark code paths. Note that while the correctness feedback check ensures validity, it does not guarantee that the generated benchmark captures the targeted performance-critical behaviors.

3 Study Setup

To evaluate the performance of *LLM4JMH*, we conduct an empirical study across real-world Java projects. This section details the studied subjects, execution environment, and benchmark configuration to ensure reproducibility and practical relevance.¹

3.1 Studied Subjects

In this study, we select three real-world Java projects from different domains. The selected projects include *RxJava* [38], *Eclipse Collections* [13], and *Zipkin* [59]. These systems are chosen to reflect a broad spectrum of runtime characteristics, e.g., event-driven concurrency (*RxJava*), memory-intensive data structure operations (*Eclipse Collections*), and I/O-heavy distributed tracing workloads (*Zipkin*). This selection enables us to evaluate the effectiveness and stability of LLM-generated microbenchmarks across varied architectural patterns and performance bottlenecks. The heterogeneity of these subjects ensures that our empirical findings are robust and generalizable to a wide range of software engineering contexts. Detailed metadata, including repository size, commit history, and versioning, is summarized in Table 1.

3.2 Execution Environment and Settings

We conduct our experiments using GPT OSS [14], a state-of-the-art open-source LLM that supports code and natural language generation. All interactions with LLM are performed via its official API, with parameters set to their default values. Specifically, we set the temperature to 0 to ensure deterministic output, thereby enhancing the reproducibility of our results.

¹Replication packages: https://github.com/hitum-dev/icse26_llm4jmh_submission

To ensure consistency across runs and resource isolation, each JMH microbenchmark is executed on a dedicated VM, effectively providing an exclusive allocation equivalent to 4 CPU cores and 16 GB of memory per microbenchmark. Instances are distributed across multiple zones in **us-central1** to increase capacity and reduce scheduling contention. All benchmarks run on Ubuntu 22.04 LTS (Jammy Jellyfish) with OpenJDK 21. This setup ensures that each benchmark runs in isolation, minimizing interference from other processes and enabling stable and reproducible measurements. In total, our experiment consists of 55,527 microbenchmarks, each comprising 5 warm-up iterations of 500ms and 30 measurement iterations, with each iteration lasting 1 second.

4 Study Analysis

To systematically study the potential of LLMs in assisting software practitioners with early-stage performance regression detection, we aim to address four research questions.

4.1 RQ1: Can LLM-generated microbenchmarks provide stable and reliable performance regression detection?

4.1.1 Motivation. Microbenchmarks are widely adopted in performance engineering for their fine-grained sensitivity to system-level changes. However, their effectiveness in detecting performance regressions depends critically on *statistical stability*. Microbenchmarks with high execution variability can trigger *false positives*, reporting regressions where none exist, due to noise, non-deterministic runtime behavior, or system fluctuations. Such spurious alerts not only erode developer trust but also waste CI/CD resources on unnecessary investigations. Thus, before using LLM-generated microbenchmarks in performance regression detection workflows, it is crucial to ask: **Are these microbenchmarks trustworthy and consistent enough to rely on?** To address these concerns, this research question evaluates the *statistical robustness* of LLM-generated microbenchmarks (*LLM4JMH*), in comparison to expert-written *JMH* and automated Junit-to-JMH conversion (*ju2jmh*) as baselines. We assess not only measurement variability and convergence speed but also the potential of each benchmark suite to minimize false positive detections.

4.1.2 Approach. To evaluate the *statistical stability* of benchmark throughput measurements, we compute the *Relative Standard Deviation (RSD)* [25, 26]. *RSD* captures how much variability exists relative to the average performance, enabling us to distinguish stable benchmarks from noisy ones. Formally, for a microbenchmark with a set of throughput measurements, the *RSD* is defined as $RSD = \frac{\sigma}{\mu}$, where σ is the standard deviation and μ is the mean of the measured throughput. Smaller *RSD* values indicate more stable microbenchmarks. For example, an *RSD* of 1% implies that the measurement variability is only 1% of the average throughput, which is stable enough for detecting performance bugs [26]. To reduce the impact of outliers, we report a robust variant of *RSD*, defined as $RSD = \frac{1.4826 \times MAD}{median}$, where *MAD* denotes the median absolute deviation of the throughput measurements [31]. Throughout the experiments, we use this robust formulation as our default *RSD* metric unless otherwise stated. To mitigate JVM warm-up effects,

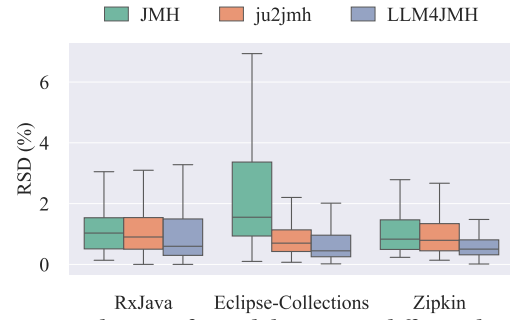


Figure 2. Distribution of variability across different benchmark suites, using *RSD* (%).

we perform five warm-up iterations of 500ms each. Afterward, each microbenchmark is executed in 30 measurement iterations, each lasting 1 second, during which throughput (operations per second) is recorded. The *RSD* is then computed from these 30 post-warm-up measurements. We consider microbenchmarks with *RSD* values above 5% as unstable, as they are less suitable for detecting subtle performance regressions [29]. Instead of using all available microbenchmarks in the codebase, we ensure a fair comparison by evaluating the stability of microbenchmarks that target the same methods across *JMH*, *ju2jmh*, and *LLM4JMH*. This eliminates coverage bias and isolates stability differences due solely to generation methods.

To assess *statistical precision* and *convergence behavior*, we also compute the *Relative Confidence Interval Width (RCIW)* [26]. *RCIW* quantifies how tight the confidence interval is relative to the mean, offering insight into measurement uncertainty across repeated runs. Formally, for a given microbenchmark, the *RCIW* at confidence level α (e.g., 99%) is computed as $RCIW = \frac{U_{CI} - L_{CI}}{\mu}$, where μ is the mean of throughput, U_{CI} and L_{CI} are the upper and lower bounds of the two-sided confidence interval for μ . In our experiments, we collect 30 measurements per microbenchmark, and use 10,000 bootstrap samples to compute a two-sided 99% confidence interval. Lower *RCIW* values indicate greater measurement stability and statistical confidence.

4.1.3 Results and Analysis. *LLM4JMH* benchmarks are substantially more stable than both expert-written *JMH* and *ju2jmh*. Figure 2 shows the distribution of *RSD* across three benchmark suites. It reveals that *LLM4JMH* microbenchmarks exhibit tighter interquartile ranges and fewer extreme outliers compared to *ju2jmh*, indicating greater statistical consistency. Notably, the result of *Eclipse Collections* demonstrates this contrast most clearly. To provide more detailed insights, Table 2 reports the *RSD* values for benchmarks partitioned by fixed thresholds: those with $RSD \leq 1\%$

Table 2. Stability analysis showing the percentage of benchmarks classified as stable ($RSD \leq 1\%$) or unstable ($RSD \geq 5\%$).

		RxJava	Eclipse Collections	Zipkin
Stable	JMH	48.16	27.25	56.86
	ju2jmh	54.80	68.81	60.91
	LLM4JMH	63.78	75.98	81.57
Unstable	JMH	4.10	14.12	0.00
	ju2jmh	3.92	0.25	0.36
	LLM4JMH	6.62	5.08	1.61

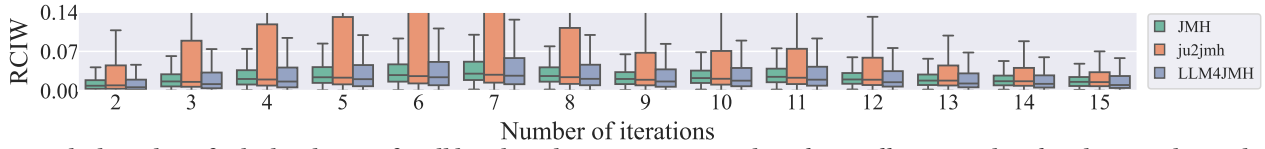


Figure 3. The box-plots of calculated *RCIW* for all benchmark suites in *RxJava*. The *Eclipse Collections* and *Zipkin* show similar tendencies. Detailed results are available in the replication packages.

are considered stable [26], while those with $RSD \geq 5\%$ are considered unstable [20, 21]. Across all systems, *LLM4JMH* consistently outperforms both expert-written *JMH* and *ju2jmh* in reducing measurement variability, especially within the stable subset. In *Eclipse Collections*, for example, the proportion of stable *LLM4JMH* benchmarks is 75.98%, which is markedly higher than the 27.25% observed in *JMH*. Similar improvements are evident in *RxJava* and *Zipkin*, where *LLM4JMH* achieves 63.78% and 81.57% stable microbenchmarks, respectively, again significantly better than the *JMH* and *ju2jmh* baselines. Even within the unstable subset, *LLM4JMH* benchmarks achieve *RSD* comparable to expert-written *JMH* tests, such as 6.62% in *RxJava* and 1.61% in *Zipkin*.

LLM4JMH benchmarks converge faster than *ju2jmh* and show comparable convergence behavior to expert-written microbenchmarks. Beyond stability, we evaluate the convergence tendencies of microbenchmarks using the *RCIW*. As shown in Figure 3, *LLM4JMH* benchmarks converge more quickly than *ju2jmh*, with *RCIW* values converging below 0.03 by the 10th iteration in most settings. This behavior is particularly important in CI/CD pipelines, where fast and trustworthy performance signals are critical. In *Eclipse Collections* and *Zipkin*, *LLM4JMH* not only reaches stability earlier but also maintains lower *RCIW* dispersion across benchmarks, demonstrating that *LLM4JMH* enables efficient and statistically meaningful measurements with limited overhead.

LLM4JMH benchmarks demonstrate a lower likelihood of false positive detections. A core concern with unstable benchmarks is the elevated likelihood of false positives, i.e., incorrectly identifying regressions due to high variance rather than genuine performance shifts. Microbenchmarks with high *RSD* or slow *RCIW* convergence exacerbate this issue. The data shows that *LLM4JMH* exhibits lower false alarm potential due to its more stable *RSD* profiles and tighter confidence bounds, whereas *ju2jmh* frequently suffers from erratic variability even under repeated execution. Further evidence presented in Section 5.5 substantiates this observation.

RQ1 Takeaway: *LLM4JMH* achieves significantly better statistical stability than automated baselines such as *ju2jmh* and achieves performance comparable to expert-written microbenchmarks.

4.2 RQ2: To what extent can domain knowledge improve the quality of LLM-generated performance microbenchmarks?

4.2.1 Motivation. While LLMs show promise in generating performance microbenchmarks, their outputs often lack awareness of key performance engineering heuristics. Expert practitioners typically avoid benchmarking overly long methods or those with

many callees, as such structures introduce noise from diverse control paths, hidden state changes, and non-deterministic behavior, making it difficult to isolate and interpret performance regressions. Instead, experts target concise, semantically meaningful methods to ensure stable and interpretable measurements, often refactoring benchmarks to focus on inner or frequently invoked routines.

This motivates a key question: *Can domain knowledge, e.g., static analysis over call graph depth and method complexity, improve the stability and diagnostic value of LLM-generated microbenchmarks?* RQ2 investigates whether employing domain knowledge such as structural properties can steer generative models toward microbenchmarks that are not only valid, but also statistically stable and practically actionable.

4.2.2 Approach. To improve the stability of LLM-generated microbenchmarks, we inject domain knowledge in the form of structural code analysis. Specifically, we empirically profile microbenchmarks generated for *RxJava*, examining two static features known to affect runtime variance: (1) the total lines of code (LOC) in the transitive call graph, and (2) the number of distinct method invocations. As shown in Figure 4, benchmarks with transitive call graphs exceeding 25 LOC or invoking five or more distinct methods exhibit substantially higher *RSD*, often exceeding 40–60%. These patterns align with expert practices in microbenchmark design, where minimal control flow and reduced execution depth are favored to ensure reproducible results.

Guided by these insights, we define conservative structural thresholds and apply them as filters to all generated benchmarks. Notably, the same thresholds, derived from *RxJava*, generalize well to *Eclipse Collections* and *Zipkin*, suggesting the effectiveness of these heuristics across systems. This filtering step forms the core of our domain knowledge selection strategy, enhancing measurement reliability.

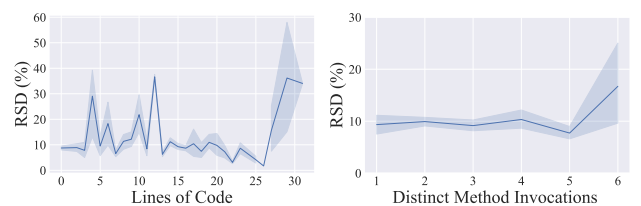


Figure 4. Analysis of static code features for LLM-generated microbenchmarks in *RxJava*: (left) distribution of LOC in transitive call graph; (right) distribution of distinct method invocations.

4.2.3 Results and Analysis. Employing domain knowledge selection into LLM-generated microbenchmarks improves their stability by reducing *RSD*.

Figure 5 and Table 3 provide a comparative analysis of the stability of LLM-generated microbenchmarks with and without domain knowledge selection (*LLM4JMH* vs. *LLM4JMH+*). The results highlight the tangible benefits of incorporating static code insights into

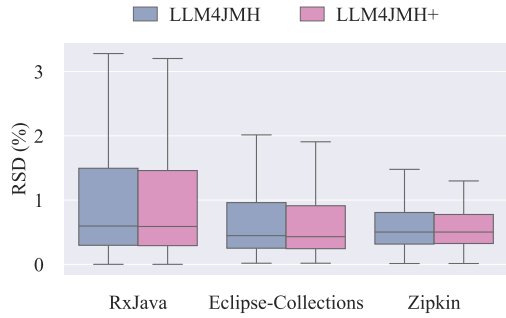


Figure 5. Impact of domain knowledge selection on the stability of LLM-generated microbenchmarks, using RSD (in %).

the microbenchmark generation process. Across all three systems, *LLM4JMH+* consistently reduces variability as measured by *RSD*. In *RxJava*, for example, the median *RSD* drops substantially, and the distribution becomes more compact with fewer high-variance outliers. Similar trends are observed in *Eclipse Collections* and *Zipkin*, underscoring the generalizability of the approach across domains.

Table 3 further quantifies the improvement by partitioning benchmarks into stable ($RSD \leq 1\%$) and unstable ($RSD \geq 5\%$) groups. In all cases, *LLM4JMH+* either improves or matches the stability of its baseline counterpart. Notably, the proportion of unstable benchmarks drops from 6.62% to 6.01% in *RxJava*, from 5.15% to 1.88% in *Eclipse Collections*, and from 3.92% to 1.88% in *Zipkin*. These reductions suggest that selection via domain knowledge mitigates structural factors that lead to high measurement noise, such as long call chains or excessive method dispatch. The stable group remains competitive or improves slightly, indicating that the structural constraints help steer generation away from inherently noisy benchmarks without restricting the diversity of microbenchmarks.

RQ2 Takeaway: Integrating domain knowledge selection for *LLM4JMH*, i.e., *LLM4JMH+*, significantly improves benchmark stability across all systems.

4.3 RQ3: How effective can LLMs assist in detecting performance regressions?

4.3.1 Motivation. An effective microbenchmarking suite must detect performance regressions with high precision, especially in CI/CD settings where throughput anomalies can indicate subtle but impactful bugs. This research question investigates whether *LLM4JMH+* can effectively and efficiently identify artificially injected performance regressions by comparing its detection capability against expert-written microbenchmarks (*JMH*) and microbenchmarks automatically converted from unit tests (*ju2jmh*).

Table 3. Stability impact of domain knowledge on LLM-generated microbenchmarks, reporting the proportion classified as stable ($RSD \leq 1\%$) or unstable ($RSD \geq 5\%$). The best results are marked in bold.

		RxJava	Eclipse Collections	Zipkin
Stable	LLM4JMH	63.78	75.98	81.57
	LLM4JMH+	64.36	77.06	83.10
Unstable	LLM4JMH	6.62	5.15	3.92
	LLM4JMH+	6.01	1.88	1.88

Table 4. Detection effectiveness and capability of benchmark suites.

Subject	Benchmark Suites	Mutation Sensitivity					Mutation Score				
		1%	5%	10%	50%	90%	1%	5%	10%	50%	90%
RxJava	JMH	29.99	26.01	24.80	23.08	17.88	100.00	100.00	100.00	82.61	82.61
	ju2jmh	65.67	63.56	62.35	55.34	41.86	100.00	100.00	100.00	100.00	91.30
	LLM4JMH+	59.94	51.24	49.24	41.22	26.83	100.00	100.00	100.00	86.96	82.61
Eclipse Collections	JMH	28.85	19.92	13.77	2.70	0.00	81.48	81.48	81.48	62.96	0.00
	ju2jmh	83.61	67.65	49.58	10.50	0.84	81.48	74.07	70.37	33.33	7.40
	LLM4JMH+	95.73	95.28	94.55	69.42	1.18	81.48	81.48	81.48	81.48	33.33
Zipkin	JMH	45.45	41.11	35.57	32.41	27.67	81.25	81.25	81.25	68.75	68.75
	ju2jmh	49.92	45.84	42.43	36.41	29.96	93.75	93.75	87.50	87.50	87.50
	LLM4JMH+	45.45	45.45	45.45	42.42	42.42	87.50	62.50	43.75	37.50	37.50

Mutation sensitivity: percentage of microbenchmarks that detect at least one mutant. **Mutation score:** percentage of mutants detected by at least one microbenchmark. Threshold columns (1%, 5%, etc.) denote the minimum performance regressions δ required for detection. Higher values indicate stronger detection effectiveness and broader bug detection capability.

4.3.2 Approach. To evaluate the effectiveness of *LLM4JMH+* in detecting performance regressions, we simulate synthetic faults via controlled mutation and assess detection effectiveness, capability, and efficiency across benchmark suites.

Evaluation Pipeline. We establish a rigorous pipeline to ensure fair and comparable evaluation across three benchmark suites, i.e. *JMH*, *ju2jmh*, and *LLM4JMH+*.

Step 1: Identifying intersections of methods. We begin by identifying the intersection of methods exercised across all three suites to guarantee that the injected performance bugs are accessible and testable. For each method, we select only the benchmark cases that invoke it directly or transitively, thereby eliminating irrelevant executions and ensuring targeted analysis. We then inject performance regressions using a lightweight mutation strategy, specifically inserting a heavy-weight operator (HWO) (*Thread.sleep(0, 1)*) to simulate minor slowdowns. We use *Thread.sleep(0, 1)* as it introduces the smallest portable delay in Java (i.e., 1 nanosecond) that cannot be optimized away yet accumulates into measurable overhead on hot paths. This provides a stable, cross-platform way to emulate micro-regressions. If benchmarks can detect this tiny slowdown, larger regressions will be even more easily observable. Finally, each benchmark suite is executed on both the original and mutated versions of the system, and we assess their ability to detect regressions based on statistical differences in throughput.

Step 2: Determining mutation targets. To determine mutation targets, we profile invocation frequencies of common methods across benchmark suites. Because the same method can be exercised by different numbers of microbenchmarks in each suite, we sort methods by hit count, prioritizing *JMH*, then *LLM4JMH+*, and finally *ju2jmh*. We apply a stratified sampling strategy across the hit count distribution to avoid bias toward frequently invoked methods while ensuring diversity in code coverage. Mutation targets are selected from the intersection of methods covered by all benchmark suites, ranked by their hit counts in expert-written *JMH* tests, and distributed into 30 stratified buckets using an equal step size of

Table 5. Average, worst-case, and total detection times (in seconds) required to uncover performance bugs at a 1% degradation threshold.

Benchmark Suites	RxJava			Eclipse Collections			Zipkin		
	Worst	Average	Total	Worst	Average	Total	Worst	Average	Total
JMH	4,947	107	7,020	11,155	112	15,633	1,043	70	1,853
ju2jmh	86,780	49	252,687	142,203	38	867,425	8,301	64	16,510
LLM4JMH+	1,321	54	3,217	470	33	10,237	1,273	70	2,275

Lower values indicate higher efficiency. Best results are marked in bold.

[max(hit)/30]. Buckets containing no eligible methods naturally reduce the total number of mutations, yielding 23, 25, and 16 final targets for *RxJava*, *Eclipse Collections*, and *Zipkin*, respectively.

Step 3: Quantifying performance regressions. To quantify the severity of performance regressions introduced by synthetic faults, we define the *bug size* using the upper bound of the confidence interval of the throughput ratio, calculated via nonparametric bootstrap resampling [18]. Let $\mathcal{B}$ be a set of microbenchmarks that invoke the mutated method, either directly or transitively. Let o and m represent the original and mutated versions of the system, respectively. Given a microbenchmark $b \in \mathcal{B}$, we execute b on both versions and the *bug size* is defined by $\text{bug_size}(m, o; b) = 1 - U_{RCI}(b, m; o)$, where $U_{RCI}(b, m; o)$ denotes the upper bound of the 99% confidence interval for the throughput ratio $RCI_b = \mu_b^m / \mu_b^o$, and μ_b^m , μ_b^o are the average throughputs of benchmark b on the mutated and original systems, respectively. The confidence interval is computed using bootstrap resampling with 10,000 iterations [18]. A microbenchmark b is said to detect (kill) the mutation if the bug size exceeds a performance degradation threshold δ , i.e., $\text{bug_size}(m, o; b) > \delta$. A higher bug size indicates a more severe performance regression exposed by the benchmark. This corresponds to a statistically significant performance drop with 99% confidence, as estimated by the upper bound of the throughput ratio's bootstrap confidence interval.

Step 4: Evaluating the effectiveness and capability of different benchmark suites. To evaluate the effectiveness and capability of benchmark suites in detecting performance regressions, we adopt two complementary metrics: *mutation sensitivity* and *mutation score*, both parameterized by the performance degradation threshold δ . *Detection effectiveness* reflects how frequently individual microbenchmarks are able to reveal performance faults. We quantify this using *mutation sensitivity*, defined as the proportion of benchmarks in a suite that detect the mutant. A higher sensitivity indicates that bug-detecting behavior is broadly distributed across the suite, which is desirable for robustness. *Detection capability*, on the other hand, captures the overall bug coverage of the benchmark suite, i.e., whether all injected performance regressions can be detected. This is measured by the *mutation score*, defined as the percentage of mutants that are detected by at least one microbenchmark. A higher mutation score implies that the suite is capable of covering a wider range of performance regressions, even if detection is concentrated in a small number of tests.

Step 5: Assessing detection efficiency. To assess detection efficiency, we first estimate the number of benchmark executions required to uncover a regression using an analytical model. We define the detection rate r as the mutation sensitivity, i.e., the proportion of microbenchmarks in $\mathcal{B}$ that detect at least one mutant in $\mathcal{M}$ under threshold δ . This benchmark-centric probability quantifies the likelihood that a randomly selected benchmark will reveal a performance bug. Assuming benchmarks are selected uniformly at random without replacement, the expected number of executions needed to detect a regression follows a negative hypergeometric distribution, i.e. $\text{expected detection effort} = \frac{|\mathcal{B}|+1}{|\mathcal{B}| \times r + 1}$, where $|\mathcal{B}|$ is the total number of microbenchmarks. This reflects the average number of executions needed to encounter a successful test under random selection without replacement. To model the worst-case

scenario, where all ineffective microbenchmarks precede any effective one, we define *worst-case detection effort* = $|\mathcal{B}| \times (1 - r) + 1$. In addition, we introduce maximal detection effort, which corresponds to executing *all* microbenchmarks in the suite, regardless of whether any bug is detected. This upper bound is especially relevant in real-world CI/CD workflows, where regressions are rare and all microbenchmarks may be executed to validate performance stability. Although the *best-case detection effort* is trivially 1 (i.e., detection on the first benchmark execution), it is only meaningful when at least one microbenchmark satisfies the detection criterion. If no microbenchmark can detect any mutant for a given threshold δ , then all detection effort metrics become undefined. In our analysis, we report effort metrics only when detection is achievable. To obtain detection latency, we multiply the expected and worst-case detection effort by the average runtime of a microbenchmark. This yields practical estimates of average and worst-case detection time. These estimates enable comparison of not just detection effectiveness but also time efficiency across benchmark suites, offering insight into their utility under resource-constrained settings. Together, the expected detection time, worst-case detection time, and total execution time, offer a comprehensive view of a suite's efficiency under both bug-present and bug-absent scenarios.

4.3.3 Results and Analysis. *LLM4JMH+* exhibits strong effectiveness in detecting subtle performance regressions, outperforming *JMH* and approaching or exceeding *ju2jmh* in *RxJava* and *Eclipse Collections*. However, their sensitivity diminishes in *Zipkin*. Table 4 summarizes the percentage of microbenchmarks that successfully identify injected performance faults across varying thresholds. For *RxJava*, *LLM4JMH+* microbenchmarks detect 59.94% of mutants at the 1% degradation level, outperforming *JMH* (29.99%) and approaching the performance of *ju2jmh* (65.67%). In *Eclipse Collections*, *LLM4JMH+* achieves the highest detection rate across all thresholds, identifying over 95% of mutants at 1% degradation. These findings confirm that *LLM4JMH+* possesses high sensitivity to subtle regressions, particularly in computation-intensive systems. However, in *Zipkin*, a system dominated by asynchronous I/O and tracing logic, the detection rates plateau near 45% for all benchmark suites. This suggests that throughput-based microbenchmarks may face inherent limitations when targeting control-heavy or latency-sensitive code paths.

LLM4JMH+ achieves the most efficient performance regression detection across all systems, significantly reducing both detection latency and total execution time compared to *JMH* and *ju2jmh*. As such, it is well-suited for CI pipelines, offering fast feedback with minimal overhead. Table 5 compares the efficiency of the three benchmark suites, i.e., *JMH*, *ju2jmh*, and *LLM4JMH+*, in detecting injected performance bugs under a 1% degradation threshold. The evaluation includes average detection latency, worst-case detection latency, and total execution time, reflecting practical costs in CI pipelines.

Across all systems, *LLM4JMH+* consistently yields the low detection latency, achieving an effective balance between precision and runtime overhead. In *RxJava*, it reduces worst-case detection time to 1,321 seconds, outperforming *JMH* (4,947s) and significantly

surpassing *ju2jmh* (86,780s). Average detection time is also competitive, with *LLM4JMH+* needing only 54 seconds, versus 107s (*JMH*) and 49s (*ju2jmh*). The total time for running all microbenchmarks is also lowest (3,217s), demonstrating its advantage when no bug is present. In *Eclipse Collections*, *LLM4JMH+* again leads with the shortest worst-case (470s) and average (33s) detection times, and its total execution time (10,237s) is far lower than *ju2jmh*'s extremely high 867,425s and *JMH*'s 15,633s. In *Zipkin*, a latency-sensitive, IO-bound system, all benchmark suites show somewhat reduced performance. However, *LLM4JMH+* still provides competitive efficiency: 70s average and 1,273s worst-case detection latency, comparable to *JMH* (70s / 1,043s) and *ju2jmh* (64s / 8,301s). The total execution time remains favorable at 2,275s, lower than *ju2jmh* (16,510s) and slightly above *JMH* (1,853s), illustrating the strength of *LLM4JMH+* under practical runtime constraints.

These results confirm that *LLM4JMH+* offers a compelling advantage in detection efficiency, minimizing time-to-detection and total test overhead, both of which are crucial in continuous integration environments. Its efficient microbenchmark targeting and non-redundant design directly contribute to faster regression surfacing with fewer executions.

RQ3 Takeaway: *LLM4JMH+* achieves comparable or superior detection rates while drastically reducing detection latency and total runtime.

4.4 RQ4: Can generated performance microbenchmarks detect real-world performance regressions?

4.4.1 Motivation. While evaluation of mutation-based performance regressions (RQ3) offers a controlled setting for measuring microbenchmark sensitivity, it is crucial to validate whether *LLM4JMH+* can detect genuine performance regressions occurring in real-world development workflows. Real-world performance regressions often stem from subtle inefficiencies tied to evolving APIs, concurrency, and architectural changes, posing different challenges than synthetic mutations.

4.4.2 Approach. To assess real-world applicability, we curate 11 developer-confirmed performance regressions from the *Apache Flink* project [3]. We identify these bugs using the Apache JIRA tracker² by issuing the following JQL query:

```
project = FLINK AND (summary ~ "performance regression" OR
description ~ "performance regression" OR labels in
("performance", "perf", "slow", "slowdown"))
```

We retain only issues with clear, reproducible performance degradation and corresponding patches.

For each selected issue, we construct two system snapshots, i.e., the *mutant version* is the commit that introduces the performance regressions, and the *baseline version* is the commit immediately prior to the mutant one. For example, the performance regression is introduced by FLINK-16535 at commit #d7525ba, and the prior version at commit #a267019 is considered as the baseline one.

We then reuse the methodology from RQ3: run the generated benchmark suite 30 times on both versions, compute the throughput

²<https://issues.apache.org/jira/projects/FLINK/issues/FLINK-36975?filter=allopenissues>

Table 6. Detection effectiveness and capability in real-world performance regressions in *Apache Flink*.

Mutation Sensitivity					Mutation Score				
1%	5%	10%	50%	90%	1%	5%	10%	50%	90%
32.34	18.56	3.27	0.00	0.00	54.54	33.24	5.24	0.00	0.00

ratio using RCI (Ratio of Means) with 10,000 bootstrap iterations, and identify a performance regression if the upper bound of the 99% confidence interval (U_{RCI}) is below $1 - \delta$, where δ is the predefined bug size threshold.

4.4.3 Results and analysis. *LLM4JMH+* effectively detects real-world performance regressions, identifying over 50% of confirmed cases at low degradation thresholds. Table 6 summarizes the detection capabilities of *LLM4JMH+* on real-world performance regressions in *Apache Flink*, using both *mutation sensitivity* and *mutation score*. At a 1% performance degradation threshold, 32.34% of microbenchmarks detected at least one regression, while 54.59% of injected regressions were caught by some microbenchmarks. However, detection drops rapidly at higher performance degradation: only 1.27% of microbenchmarks catch 10% regressions, and none detect 50% or above.

Real-world performance regression involves heavy-weight operations. To examine whether real-world bugs share structural similarities with the synthetic HWOs introduced in prior RQs, we manually analyzed the root causes of two representative regressions, FLINK-17799 and FLINK-32685, as shown in Figure 6. In both cases, the performance degradation stemmed from repeated, unguarded invocation of expensive, stateful components in performance-critical paths. For example, FLINK-17799 repeatedly called `inputGate.requestPartitions()`, involving virtual dispatch, synchronization, and memory access. The patch resolved the issue by introducing a local flag to prevent redundant calls, yielding a 4.5x throughput improvement. Similarly, FLINK-32685 introduced changelog-related logic within a latency-sensitive mailbox loop, resulting in cumulative slowdown. The fix involved conditional bypassing of unnecessary metrics updates. These findings confirm that HWOs, i.e., methods with non-trivial side effects and large execution footprints, are a common source of real-world performance regressions.

What we learned from undetected regressions. We manually inspect the real-world regressions undetected by *LLM4JMH+*, revealing structural blind spots in our approach. First, regressions arise from cross-module or feedback-driven interactions, such as mailbox scheduling and adaptive throughput control in FLINK-14304 and FLINK-23451, where slowdowns appear only under concurrent or long-running execution that file-level benchmarks cannot capture. Second, regressions such as FLINK-19010 and FLINK-25511 involve resource lifecycle effects, including checkpoint recovery and materialization across task restarts, which fall outside the short steady-state window of microbenchmarks. Third, fixes like the batching optimization in FLINK-30533 address fine-grained call stack overheads visible only through end-to-end pipeline profiling. Thus, *LLM4JMH+* reliably detects local inefficiencies yet struggles

```

--- flink-runtime/.../AbstractRecordReader.java
+++ flink-runtime/.../AbstractRecordReader.java
@@ -66,7 +68,10 @@ protected boolean getNextRecord(T target) throws IOException,
  ↳ InterruptedException
  // The action of partition request was removed from InputGate#setup since
  ↳ FLINK-16536, and this is the only
  // unified way for launching partition request for batch jobs. In order to avoid
  ↳ potential performance concern,
  // we might consider migrating this action back to the setup based on some
  ↳ condition judgement future.
- inputGate.requestPartitions();
+ if (!requestedPartitions) {
+   inputGate.requestPartitions();
+   requestedPartitions = true;
+ }

```

Figure 6. (left) Fix for FLINK-17799: avoids repeated partition requests in batch jobs. (right) Fix for FLINK-32685: prevents redundant changelog operations in the mailbox loop.

Table 7. Anti-pattern analysis. The anti-pattern rate is defined as the number of detected cases in each anti-pattern category divided by the total number of benchmarks in the suite.

Subject	Benchmark Suites	RETU	LOOP	FINAL	INVO	FORK	Overall
RxJava	JMH	0.92	6.91	0.00	0.00	0.00	6.45
	LLM4JMH	4.14	2.86	0.43	0.00	0.00	7.43
Eclipse Collections	JMH	16.31	18.62	0.58	0.00	0.00	35.51
	LLM4JMH	7.15	3.30	0.09	0.00	0.00	10.54
Zipkin	JMH	8.22	0.00	0.00	0.00	0.00	8.22
	LLM4JMH	5.71	1.14	0.00	0.00	0.00	6.86

The "Overall" column reports aggregated anti-pattern rate across all categories for each benchmark suite. Values are given in percentages (%) with emergent system-level regressions driven by asynchronous control or cumulative state.

RQ4 Takeaway: *LLM4JMH+* demonstrates strong practical utility by detecting over **50% of real-world performance regressions** at low degradation thresholds ($\delta = 1\%$), confirming the suitability for CI/CD-based performance regression monitoring.

5 Discussion

This section synthesizes our empirical findings and examines how LLM-generated microbenchmarks behave across multiple dimensions of quality.

5.1 Microbenchmark Anti-Patterns Analysis

To examine whether LLM-generated microbenchmarks exhibit known JMH anti-patterns (e.g., dead-code elimination, constant folding, or missing benchmark state isolation), we used SpotJMH-Bugs [8], a static analysis tool designed to identify five common JMH anti-pattern categories (i.e., RETU, LOOP, FINAL, INVO, FORK). Since SpotJMH-Bugs reports false positives, two authors with JMH background jointly conducted a manual validation. The authors independently labeled each detected result from SpotJMH-Bugs, achieving Cohen’s kappa coefficient [34] of 91%, and any disagreements were resolved by discussion.

LLM4JMH exhibits anti-pattern rates that are comparable to expert-written JMH benchmarks. As shown in Table 7, *LLM4JMH* shows fewer anti-patterns in *Eclipse Collections*, with much lower RETU, LOOP, and FINAL rates, resulting in an overall rate of 10.54% compared to 35.51% in the expert-written *JMH*. In *Zipkin*, *LLM4JMH* likewise produces lower RETU (5.71% vs. 8.22%) and only minor LOOP misuse, yielding a slightly lower overall rate

```

--- a/flink-streaming-java/.../StreamTask.java
+++ b/flink-streaming-java/.../StreamTask.java
@@ -596,7 +596,8 @@ protected void processInput(MailboxDefaultAction.Controller
  ↳ controller) throws E
  } else if (!inputProcessor.isAvailable()) {
    timer = new GaugePeriodTimer(ioMetrics.getIdleTimeMsPerSecond());
    resumeFuture = inputProcessor.getAvailableFuture();
- } else if (changelogWriterAvailabilityProvider != null) {
+ } else if (changelogWriterAvailabilityProvider != null
+   && !changelogWriterAvailabilityProvider.isAvailable()) {
  // waiting for changelog availability is reported as busy
  timer = new GaugePeriodTimer(ioMetrics.getChangelogBusyTimeMsPerSecond());
  resumeFuture = changelogWriterAvailabilityProvider.getAvailableFuture();

```

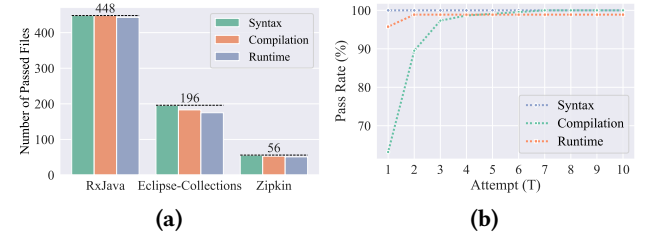


Figure 7. Effectiveness and convergence of correctness feedback. (a) Number of generated benchmarks passing syntax, compilation, and runtime validation across subjects. (b) Pass-rate improvement over up to ten feedback-guided repair attempts (T), showing convergence for *RxJava*; *Eclipse Collections* and *Zipkin* exhibit similar trends. Detailed results are in the replication packages.

(6.86% vs. 8.22%). For *RxJava*, *LLM4JMH* has higher RETU (4.14% vs. 0.92%) but fewer LOOP cases (2.86% vs. 6.91%), with FINAL and INVO remaining rare in both suites. Overall, *LLM4JMH* achieves structural correctness on par with expert-written microbenchmarks, and in some projects, shows fewer common anti-pattern categories.

5.2 Impact of Correctness Feedback Check

To quantify the effectiveness of our correctness feedback check, we define two key metrics: (1) *Passed Files* denote generated benchmark classes (one per source file) in which all emitted microbenchmark methods successfully satisfy a given validation stage (syntax, compilation, or runtime). (2) *Pass Rate* is the proportion of benchmark classes that pass a specific stage. For example, a 90% runtime pass rate indicates that 90% of generated classes execute without errors, with all contained benchmarks running successfully.

The correctness feedback check is capable of filtering out invalid benchmarks, ensuring that only fully runnable microbenchmarks reach evaluation. Figure 7a reports the number of passed files across the three subjects. For *RxJava*, nearly all 448 generated benchmarks pass all stages. In contrast, *Eclipse Collections* and *Zipkin* show sharp drop-offs from syntax to runtime (196 and 56 valid files, respectively), indicating that syntactic and compilation correctness alone does not ensure semantic executability. This highlights the importance of the correctness feedback check in producing benchmarks that yield meaningful performance data for regression detection.

Feedback-guided repair converges quickly, producing executable benchmarks within a small number of refinement steps. Figure 7b shows how pass rates evolve with repeated repair

Table 8. Detection effectiveness and capability of benchmark suites across multiple LLMs.

Subject	Benchmark Suites	Mutation Sensitivity					Mutation Score				
		1%	5%	10%	50%	90%	1%	5%	10%	50%	90%
RxJava	GPT OSS (120B)	62.79	57.26	54.62	44.47	36.35	100.00	100.00	100.00	100.00	100.00
	Gemini 2.0 Flash	74.25	70.58	68.86	59.10	46.01	100.00	100.00	100.00	100.00	100.00
	Qwen 2.5 Coder (32B)	79.23	76.06	74.65	66.20	51.76	100.00	100.00	100.00	93.33	80.00
Eclipse Collections	GPT OSS (120B)	16.96	3.69	1.38	0.00	0.00	90.91	81.81	54.55	0.00	0.00
	Gemini 2.0 Flash	15.83	3.42	2.00	0.00	0.00	100.00	63.63	36.36	0.00	0.00
	Qwen 2.5 Coder (32B)	10.99	0.00	0.00	0.00	0.00	63.64	0.00	0.00	0.00	0.00
Zipkin	GPT OSS (120B)	14.75	2.95	1.64	0.00	0.00	81.82	54.55	54.55	0.00	0.00
	Gemini 2.0 Flash	21.54	5.13	1.03	0.00	0.00	81.82	36.36	18.18	0.00	0.00
	Qwen 2.5 Coder (32B)	3.45	3.45	0.00	0.00	0.00	9.09	9.09	0.00	0.00	0.00

attempts. Across all subjects, pass rates increase sharply within the first few iterations and stabilize around $T = 10$. *RxJava* achieves near-perfect pass rates (>98%) after two attempts, while *Eclipse Collections* and *Zipkin* rise from 40–60% to above 90% (see replication packages). These results validate that iterative refinement efficiently transforms raw LLM output into a syntactically, semantically, and runtime-valid benchmark corpus.

5.3 Impact of different LLMs

Following the RQ3 methodology, we evaluate three commonly used LLMs: GPT-OSS, Gemini-2.0-Flash (Gemini), and Qwen-2.5-Coder (Qwen). These models are chosen based on their descending code generation capability as reported by the Aider leaderboard [1]. This allows us to assess how an LLM’s code-generation strength influences the effectiveness of *LLM4JMH*.

LLMs with stronger code-generation capability detect performance regressions more effectively. As shown in Table 8, all three models achieve near-perfect scores in *RxJava*, indicating that lower-capability models suffice when runtime behavior is simple. However, in *Eclipse Collections* and *Zipkin*, Qwen’s accuracy collapses beyond 5% slowdowns, while GPT-OSS and Gemini maintain high detection rates of up to 50% (e.g., GPT-OSS detects 54% of 10% regressions in *Zipkin* vs. Qwen’s 0%).

5.4 Qualitative Analysis of Microbenchmarks

To qualitatively understand the strengths and limitations of our approach, we conducted a manual analysis of microbenchmarks from *RxJava*, *Eclipse Collections* and *Zipkin* across our generated (by *LLM4JMH*) and expert-written benchmarks. Our analysis focused on performance regressions that were detected by one type of benchmarks but not by the other (e.g., detected by *LLM4JMH* but not by expert-written benchmarks). In total, we identified 30 mutants at the bug size of 1% and 99%. We chose these bug sizes for our qualitative study, due to them being the two extreme scenarios that correspond to the easiest and most challenging performance regressions to be detected by both approaches in our settings. For each regression, we selected one benchmark from those generated by *LLM4JMH* and one benchmark that is expert-written, based on the hitting ratio. We selected benchmarks for the qualitative analysis based on their hitting ratio, as prior work [20] has shown that benchmarks with a higher hitting ratio are more likely to detect regressions.

Two authors of this paper jointly reviewed all selected performance regressions and corresponding benchmarks to learn why

each benchmark was or was not able to detect a performance regression. We split up the selected performance regressions into two groups: synchronous functions, and asynchronous functions. Subsequently, the two authors independently annotated each performance regression and benchmark based on the learned groups and calculated the level of agreement between their labels, which resulted in a Cohen’s kappa coefficient of 93%. In cases where two authors assigned different labels, they discussed the discrepancies until reaching a consensus. We analyze the benchmarks for each regression group and demonstrate the insights in this subsection.

Synchronous functions (73.3% cases): Performance regressions in synchronous functions are likely to be detected by our *LLM4JMH* approaches. Our analysis revealed that *LLM4JMH* is particularly effective at identifying performance regressions caused by changes within the synchronous function. In contrast, expert-written benchmarks often involve multiple interacting functions, which can obscure subtle regressions within individual methods due to interprocedural noise (see replication packages). In fact, 54.54% of the performance regression cases in synchronous functions cannot be detected by expert-written benchmarks due to the involved multi-function call scenarios, where the regression effect was diluted or masked. This finding suggests that LLM-generated benchmarks, by isolating functions and minimizing extraneous interactions, are more effective at capturing fine-grained performance issues stemming from localized code changes.

Asynchronous functions (30% cases): Our analysis shows that for the easiest performance regressions to detect (1% bug size) in asynchronous functions, the benchmarks generated by *LLM4JMH* perform comparably to those written by experts (44.44% vs. 55.55%). Such asynchronous functions generally have simple implementations. However, for the most challenging performance regressions to detect (99% bug size), the expert-written benchmarks perform significantly better. These more challenging cases involve asynchronous functions with complex implementations, as illustrated in our replication packages. In this scenario, only 25% of the LLM-generated benchmarks successfully identified the regressions, compared with 75% of the expert-written benchmarks. *LLM4JMH* struggles to effectively capture the most challenging performance regressions in asynchronous functions. We conjectured that this is due to the limitations of LLMs in modeling and reasoning about concurrency primitives and asynchronous execution patterns. This highlights the importance of integrating such advanced programming knowledge into LLM-based code models.

5.5 False-Positive Rate in *LLM4JMH*

To evaluate the robustness of *LLM4JMH*, we estimate the false-positive rate (FPR) by comparing independent executions of the same, unchanged benchmark codebase. Specifically, we rerun each benchmark under identical experimental conditions, and any reported performance regression across these runs is considered a false positive. This approach captures natural execution variability, including measurement noise and runtime nondeterminism, without introducing artificial code changes.

***LLM4JMH* achieves similar false-positive rates as expert-written *JMH* and *ju2jmh*, particularly under higher regression thresholds (> 5%).** As shown in Table 9, we find that *LLM4JMH*

Table 9. False-positive rates of benchmark suites across varying performance regression δ . Best results are marked in bold.

Subject	Benchmark Suites	Mutation Sensitivity				
		1%	5%	10%	50%	90%
RxJava	JMH	9.42	2.08	0.81	0.00	0.00
	ju2jmh	26.13	3.86	0.99	0.01	0.01
	LLM4JMH	18.55	5.78	2.29	0.12	0.00
Eclipse Collections	JMH	24.34	8.91	5.21	0.22	0.00
	ju2jmh	52.14	11.18	3.11	0.01	0.00
	LLM4JMH	18.80	5.45	2.72	0.27	0.00
Zipkin	JMH	37.70	8.20	6.56	1.64	0.00
	ju2jmh	49.56	9.47	3.08	0.00	0.00
	LLM4JMH	30.56	6.74	2.38	0.00	0.00

consistently shows lower false-positive rates than *ju2jmh* under different regression thresholds from 1% to 90% in *Eclipse Collections* and *Zipkin*, indicating better stability under small, non-impactful code changes. For *RxJava*, though *LLM4JMH* has a higher false-positive rate, the difference is relatively insignificant. The slightly higher false-positive rate is likely attributable to the intrinsic characteristics of *RxJava*, which comprises a large number of fully asynchronous and event-driven code implementations, making it difficult for LLMs to understand. Across all subjects, false positives decline sharply as the regression threshold increases, demonstrating that both expert-written benchmarks and *LLM4JMH* are robust to larger performance differences. In summary, these empirical analyses demonstrate that *LLM4JMH* maintains low false-positive rates, indicating strong reliability in distinguishing harmless changes from genuine performance differences.

6 Threats to Validity

External Validity. Our study focuses on three well-known Java systems that heavily utilize JMH and JUnit. While representative of real-world performance engineering, the results may not generalize to other domains, languages, or architectures. We also simulate regressions using a heavy-weight mutation operator to inject delay-based slowdowns. Although HWOs are grounded in prior work [10, 20, 24], they do not cover all regression types. Expanding the mutation set is a goal for future work. In addition, our approach employs a file-level generation strategy that scales well operationally, as only changed files require regeneration and most commits affect only a few files. While the generation overhead is negligible, the execution phase of microbenchmarks remains the dominant contributor to end-to-end cost.

Internal Validity. Benchmarking accuracy may be influenced by system noise or unstable runtime conditions. We mitigate this through standard best practices: CPU pinning, warmups, and repeated trials. LLM generation randomness also introduces variation; different prompts may yield functionally distinct benchmarks. Future work may explore decoding constraints or ensemble prompting for more stable output. We define detection thresholds (e.g., $\geq 1\%$, $\geq 5\%$) and these thresholds can lead to different sets of regressions, which could influence the robustness of our findings.

Construct Validity. We rely on throughput degradation and static structural features (e.g., call graph LOC, callee count) to estimate benchmark quality and regression severity. While empirically justified, these proxies do not fully capture system-level non-determinism (e.g., GC, I/O). Incorporating runtime profiling may improve fidelity in future evaluations. In addition, domain-specific

knowledge (e.g., JMH best practices and anti-pattern rules) is currently applied post-hoc during repair, which may introduce bias or discard valid candidates. Future work will explore integrating such knowledge directly into the generation phase to reduce repair bias and improve benchmark correctness.

7 Related Work

Microbenchmarks and Performance Testing. Microbenchmarks play a crucial role in detecting fine-grained performance regressions that often elude traditional functional tests [16, 22]. Tools such as JMH [37] have become the de facto standard for Java performance benchmarking. However, they require developers to manually craft and maintain low-level tests, a process that is labor-intensive, error-prone, and typically incomplete in large or rapidly evolving codebases. To make performance microbenchmarks more accessible by software developers, prior work has begun to explore automated approaches for generating JMH microbenchmarks directly from source code and unit tests [20]. Moreover, recent studies [6, 12, 28] highlight the lack of systematic performance testing in CI/CD pipelines, emphasizing the need for automated and reliable solutions. While frameworks like JMH provide the infrastructure for running benchmarks, the challenge of generating meaningful and stable microbenchmarks remains largely unaddressed.

Automated Test Generation. The field of automatic test generation has traditionally focused on functional correctness [9, 15], with techniques ranging from symbolic execution to search-based methods. Traditional tools such as EvoSuite [15] and Randoop [39] are widely used on Java code bases. Recent advances in LLMs [52] have expanded the scope of automated testing, enabling tasks such as unit test generation [2, 5, 27, 30, 42, 47, 49, 54], test oracle creation [33, 36, 51], and system-level test input synthesis [11, 50, 55, 56]. For instance, LLMs have been fine-tuned or prompted to generate unit tests that validate individual code components, often outperforming traditional tools like EvoSuite in certain scenarios. Test oracle generation, particularly assertion synthesis, has also benefited from LLMs, which can infer expected outcomes from code context. These works demonstrate that LLMs can reduce manual effort while maintaining or improving test coverage. However, the existing efforts predominantly target logical correctness and lack the specialized requirements of performance testing, such as minimizing runtime variance or simulating realistic workloads.

Mutation Testing for Performance. Mutation testing is widely used to evaluate the adequacy of functional test suites [40]. Prior work has introduced performance-specific mutation operators, such as injecting delays or memory leaks, to simulate real-world regressions [4, 32, 53, 57]. For example, Delgado-Perez et al. [10, 46] propose performance mutation testing but rely on manually written benchmarks. These approaches typically assume the presence of existing tests and do not tackle the challenge of automatically generating performance-sensitive tests from scratch.

Unlike prior work, we automatically generate reliable performance microbenchmarks by combining LLM-based generation with domain-specific structural filtering. Our method targets performance regression detection by enforcing measurement stability and covering meaningful execution paths. We further extend mutation testing with heavyweight operators to evaluate the sensitivity

of generated tests. This hybrid approach bridges LLM capabilities with established engineering practices, enabling scalable, CI/CD-friendly performance regression detection.

8 Conclusion

In this paper, we presented a first study of generating performance microbenchmarks using large language models (LLMs). By combining LLM-based test synthesis, a correctness-feedback loop, and static structural filtering based on call graph paths and structural features, our approach discards unstable or overly complex tests that could produce misleading results. Our empirical evaluation on three real-world Java systems shows that these benchmarks maintain low measurement noise while detecting subtle performance regressions that manual and conversion-based suites often miss. This balance of coverage and stability makes the method practical for proactive detection in CI/CD pipelines. Our results show the potential of combining generative AI with expert knowledge to make microbenchmark creation more scalable, maintainable, and effective for early-stage performance regression detection.

Acknowledgments

We thank the anonymous reviewers for their constructive comments. This work received funding from the Federal Ministry of Research, Technology and Space (BMFTR) and the State of Berlin under grant number 16DII141 (Weizenbaum Institute) and by the German Research Foundation (DFG) under project number 460234259 (NFDI4DataScience).

References

- [1] Aider. 2025. Aider. <https://aider.chat/docs/leaderboards>. Accessed: 2025-11-12.
- [2] Saranya Alagarsamy, Chakkrit Tantithamthavorn, and Aldeida Aleti. 2024. A3Test: Assertion-augmented automated test case generation. *Information and Software Technology* 176, Article 107565 (2024), 15 pages. doi:10.1016/j.infsof.2024.107565
- [3] Apache. 2024. Flink. <https://flink.apache.org/>. Accessed: 2025-07-18.
- [4] Shivnath Babu, Herodotos Herodotou, et al. 2013. Massively parallel databases and MapReduce systems. *Foundations and Trends in Databases* 5, 1 (2013), 1–104. doi:10.1561/19000000036
- [5] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. 2023. CodeT5: Code generation with generated tests. In *The Eleventh International Conference on Learning Representations (ICLR '23)*. 19 pages. <https://openreview.net/forum?id=ktw68Cmu9c>
- [6] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. arXiv:2107.03374 [cs.LG] doi:10.48550/arXiv.2107.03374
- [7] Cliff Click. 2012. The Art of (Java) Benchmarking. In *Presentation at Jfokus 2012 Conference*. https://www.jfokus.se/jfokus12/preso/jf12_TheArtOfJavaBenchmarking.pdf
- [8] Diego Costa, Cor-Paul Bezemer, Philipp Leitner, and Artur Andrzejak. 2019. What's wrong with my benchmark results? Studying bad practices in JMH benchmarks. *IEEE Transactions on Software Engineering* 47, 7 (2019), 1452–1467. doi:10.1109/TSE.2019.2925345
- [9] Marcelo d'Amorim, Carlos Pacheco, Tao Xie, Darko Marinov, and Michael D. Ernst. 2006. An empirical comparison of automated generation and classification techniques for object-oriented unit testing. In *21st IEEE/ACM International Conference on Automated Software Engineering (ASE '06)*, 59–68. doi:10.1109/ASE.2006.13
- [10] Pedro Delgado-Pérez, Ana Belén Sánchez, Sergio Segura, and Inmaculada Medina-Bulo. 2021. Performance mutation testing. *Software Testing, Verification and Reliability* 31, 5, Article e1728 (2021), 24 pages. doi:10.1002/stvr.1728
- [11] Yinlin Deng, Chunqiu Steven Xia, Haoran Peng, Chenyuan Yang, and Lingming Zhang. 2023. Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '23)*. 423–435. doi:10.1145/3597926.3598067
- [12] Zishuo Ding, Jinfu Chen, and Weiye Shang. 2020. Towards the use of the readily available tests from the release pipeline as performance tests: Are we there yet?. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE '20)*, 1435–1446. doi:10.1145/3377811.3380351
- [13] Eclipse. 2024. Eclipse collections project. <https://github.com/eclipse-collections>. Accessed: 2025-07-18.
- [14] OpenAI et al. 2025. gpt-oss-120b & gpt-oss-20b model card. arXiv:2508.10925 [cs.CL] doi:10.48550/arXiv.2508.10925
- [15] Gordon Fraser and Andrea Arcuri. 2011. EvoSuite: Automatic test suite generation for object-oriented software. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE '11)*, 416–419. doi:10.1145/2025113.2025179
- [16] Andy Georges, Dries Buytaert, and Lieven Eeckhout. 2007. Statistically rigorous Java performance evaluation. *ACM SIGPLAN Notices* 42, 10 (2007), 57–76. doi:10.1145/1297105.1297033
- [17] Martin Grambow, Denis Kovalev, Christoph Laaber, Philipp Leitner, and David Bernbach. 2022. Using microbenchmark suites to detect application performance changes. *IEEE Transactions on Cloud Computing* 11, 3 (2022), 2575–2590. doi:10.1109/TCC.2022.3217947
- [18] Tim C. Hesterberg. 2015. What teachers should know about the bootstrap: Resampling in the undergraduate statistics curriculum. *The American Statistician* 69, 4 (2015), 371–386. doi:10.1080/00031305.2015.1089789
- [19] Peng Huang, Xiao Ma, Dongcai Shen, and Yuanyuan Zhou. 2014. Performance regression testing target prioritization via performance risk analysis. In *Proceedings of the 36th International Conference on Software Engineering (ICSE '14)*. 60–71. doi:10.1145/2568225.2568232
- [20] Mostafa Jangali, Yiming Tang, Niclas Alexandersson, Philipp Leitner, Jinqiu Yang, and Weiye Shang. 2022. Automated generation and evaluation of JMH microbenchmark suites from unit tests. *IEEE Transactions on Software Engineering* 49, 4 (2022), 1704–1725. doi:10.1109/TSE.2022.3188005
- [21] Mostafa Jangali, Kundi Yao, Yiming Tang, Diego Elias Costa, and Weiye Shang. 2025. Batch execution of microbenchmarks for efficient performance testing. In *2025 IEEE Conference on Software Testing, Verification and Validation (ICST '25)*. IEEE, 603–607. doi:10.1109/ICST62969.2025.10988928
- [22] Tomas Kalibera and Richard Jones. 2013. Rigorous benchmarking in reasonable time. In *Proceedings of the 2013 International Symposium on Memory Management (ISMM '13)*. 63–74. doi:10.1145/2491894.2464160
- [23] Christoph Laaber, Harald C. Gall, and Philipp Leitner. 2021. Applying test case prioritization to software microbenchmarks. *Empirical Software Engineering* 26, 6, Article 133 (2021), 48 pages. doi:10.1007/s10664-021-10037-x

- [24] Christoph Laaber and Philipp Leitner. 2018. An evaluation of open-source software microbenchmark suites for continuous performance assessment. In *Proceedings of the 15th International Conference on Mining Software Repositories (MSR '18)*, 119–130. doi:10.1145/3196398.3196407
- [25] Christoph Laaber, Joel Scheuner, and Philipp Leitner. 2019. Software microbenchmarking in the cloud. How bad is it really? *Empirical Software Engineering* 24, 4 (2019), 2469–2508. doi:10.1007/s10664-019-09681-1
- [26] Christoph Laaber, Stefan Würsten, Harald C. Gall, and Philipp Leitner. 2020. Dynamically reconfiguring software microbenchmarks: Reducing execution time without sacrificing result quality. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '20)*, 989–1001. doi:10.1145/3368089.3409683
- [27] Shuvendu K. Lahiri, Sarah Fakhoury, Aaditya Naik, Georgios Sakkas, Saikat Chakraborty, Madanlal Musuvathi, Piali Choudhury, Curtis von Veh, Jeevana Priya Inala, Chenglong Wang, et al. 2022. Interactive code generation via test-driven user-intent formalization. arXiv:2208.05950 [cs.SE] doi:10.48550/arXiv.2208.05950
- [28] Philipp Leitner and Cor-Paul Bezemer. 2017. An exploratory study of the state of practice of performance testing in Java-based open source projects. In *Proceedings of the 8th ACM/SPEC International Conference on Performance Engineering (ICPE '17)*, 373–384. doi:10.1145/3030207.3030213
- [29] Philipp Leitner and Jürgen Cito. 2016. Patterns in the chaos – A study of performance variation and predictability in public IaaS clouds. *ACM Transactions on Internet Technology* 16, 3, Article 15 (2016), 23 pages. doi:10.1145/2885497
- [30] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K. Lahiri, and Siddhartha Sen. 2023. CodaMosa: Escaping coverage plateaus in test generation with pre-trained large language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE '23)*, 919–931. doi:10.1109/ICSE48619.2023.00085
- [31] Christophe Leys, Christophe Ley, Olivier Klein, Philippe Bernard, and Laurent Licata. 2013. Detecting outliers: Do not use standard deviation around the mean, use absolute deviation around the median. *Journal of Experimental Social Psychology* 49, 4 (2013), 764–766. doi:10.1016/j.jesp.2013.03.013
- [32] Lizhi Liao, Jinfu Chen, Heng Li, Yi Zeng, Weiye Shang, Catalin Sporea, Andrei Toma, and Sarah Sajedi. 2021. Locating performance regression root causes in the field operations of web-based systems: An experience report. *IEEE Transactions on Software Engineering* 48, 12 (2021), 4986–5006. doi:10.1109/TSE.2021.3131529
- [33] Antonio Mastropaolo, Nathan Cooper, David Nader Palacio, Simone Scalabrino, Denys Poshyvanyk, Rocco Oliveto, and Gabriele Bavota. 2022. Using transfer learning for code-related tasks. *IEEE Transactions on Software Engineering* 49, 4 (2022), 1580–1598. doi:10.1109/TSE.2022.3183297
- [34] Mary L. McHugh. 2012. Interrater reliability: The kappa statistic. *Biochemia Medica* 22, 3 (2012), 276–282. doi:10.11613/BM.2012.031
- [35] Ankit P. Navik, Mukesh A. Zaveri, Sristi Vns Murthy, and Manoj Dawarwadikar. 2015. Microbenchmark based performance evaluation of GPU rendering. In *Emerging Research in Computing, Information, Communication and Applications: ERCICA 2015, Volume 1*. Springer, 407–415. doi:10.1007/978-81-322-2550-8_39
- [36] Pengyu Nie, Rahul Banerjee, Junyi Jessy Li, Raymond J. Mooney, and Milos Gligoric. 2023. Learning deep semantics for test completion. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE '23)*, 2111–2123. doi:10.1109/ICSE48619.2023.00178
- [37] OpenJDK. 2024. Java Microbenchmark Harness (JMH). <https://github.com/openjdk/jmh>. Accessed: 2025-07-18.
- [38] OpenJDK. 2024. RxJava. <https://github.com/ReactiveX/RxJava>. Accessed: 2025-07-18.
- [39] Carlos Pacheco and Michael D. Ernst. 2007. Randoop: Feedback-directed random testing for Java. In *Companion to the 22nd ACM SIGPLAN Conference on Object-oriented Programming Systems and Applications (OOPSLA '07)*, 815–816. doi:10.1145/1297846.1297902
- [40] Mike Papadakis, Marinos Kintis, Jie Zhang, Yue Jia, Yves Le Traon, and Mark Harman. 2019. Mutation testing advances: An analysis and survey. *Advances in Computers* 112 (2019), 275–378. doi:10.1016/bs.adcom.2018.03.015
- [41] Yun Peng, Akhilesh Deepak Gotmare, Michael R. Lyu, Caiming Xiong, Silvio Savarese, and Doyen Sahoo. 2025. PerfCodeGen: Improving performance of LLM generated code with execution feedback. In *2025 IEEE/ACM Second International Conference on AI Foundation Models and Software Engineering (FORGE '25)*, 1–13. doi:10.1109/Forge66646.2025.00008
- [42] Laura Plein, Wendkūni C. Ouédraogo, Jacques Klein, and Tegawendé F. Bissyandé. 2024. Automatic generation of test cases based on bug reports: A feasibility study with large language models. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion '24)*, 360–361. doi:10.1145/3639478.3643119
- [43] Marcelino Rodriguez-Cancio, Benoit Combemale, and Benoit Baudry. 2016. Automatic microbenchmark generation to prevent dead code elimination and constant folding. In *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering (ASE '16)*, 132–143. doi:10.1145/2970276.2970346
- [44] Joel Rollny. 2018. *An Extensible Framework for Large-Scale Mining of Performance Microbenchmarks*. Master's Thesis in Computer Systems and Networks. Chalmers University of Technology and University of Gothenburg.
- [45] Christopher J. Rossbach, Jon Currey, Mark Silberstein, Baishakhi Ray, and Emmett Witchel. 2011. PTask: Operating system abstractions to manage GPUs as compute devices. In *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles (SOSP '11)*, 233–248. doi:10.1145/2043556.2043579
- [46] Ana B. Sanchez, Pedro Delgado-Perez, Sergio Segura, and Inmaculada Medina-Bulo. 2018. Performance mutation testing: Hypothesis and open questions. *Information and Software Technology* 103 (2018), 159–161. doi:10.1016/j.infsof.2018.06.015
- [47] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2023. An empirical evaluation of using large language models for automated unit test generation. *IEEE Transactions on Software Engineering* 50, 1 (2023), 85–105. doi:10.1109/TSE.2023.3334955
- [48] Weiye Shang, Ahmed E. Hassan, Mohamed Nasser, and Parminder Flora. 2015. Automated detection of performance regressions using regression models on clustered performance counters. In *Proceedings of the 6th ACM/SPEC International Conference on Performance Engineering (ICPE '15)*, 15–26. doi:10.1145/2668930.2688052
- [49] Jiho Shin, Sepehr Hashtroudi, Hadi Hemmati, and Song Wang. 2024. Domain adaptation for code model-based unit test case generation. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '24)*, 1211–1222. doi:10.1145/3650212.3680354
- [50] Maolin Sun, Yibiao Yang, Yang Wang, Ming Wen, Haoxiang Jia, and Yuming Zhou. 2023. SMT solver validation empowered by large pre-trained language models. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE '23)*, 1288–1300. doi:10.1109/ASE56229.2023.00180
- [51] Michele Tufano, Dawn Drain, Alexey Svyatkovskiy, and Neel Sundaresan. 2022. Generating accurate assert statements for unit test cases using pretrained transformers. In *Proceedings of the 3rd ACM/IEEE International Conference on Automation of Software Test (AST '22)*, 54–64. doi:10.1145/3524481.3527220
- [52] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2024. Software testing with large language models: Survey, landscape, and vision. *IEEE Transactions on Software Engineering* 50, 4 (2024), 911–936. doi:10.1109/TSE.2024.3368208
- [53] Yuanjie Xia, Lizhi Liao, Jinfu Chen, Heng Li, and Weiye Shang. 2024. Reducing the length of field-replay based load testing. *IEEE Transactions on Software Engineering* 50, 8 (2024), 1967–1983. doi:10.1109/TSE.2024.3408079
- [54] Zhuokui Xie, Yinghao Chen, Chen Zhi, Shuiguang Deng, and Jianwei Yin. 2023. ChatUniTest: A ChatGPT-based automated unit test generation tool. arXiv:2305.04764 [cs.SE] doi:10.48550/arXiv.2305.04764
- [55] Shengcheng Yu, Chunrong Fang, Yuchen Ling, Chentian Wu, and Zhenyu Chen. 2023. LLM for test script generation and migration: Challenges, capabilities, and opportunities. In *2023 IEEE 23rd International Conference on Software Quality, Reliability, and Security (QRS '23)*, 206–217. doi:10.1109/QRS60937.2023.00029
- [56] Cen Zhang, Mingqiang Bai, Yaowen Zheng, Yeting Li, Wei Ma, Xiaofei Xie, Yuekang Li, Limin Sun, and Yang Liu. 2023. Understanding large language model based fuzz driver generation. arXiv:2307.12469 [cs.CR] doi:10.48550/arXiv.2307.12469
- [57] Cen Zhang, Yaowen Zheng, Mingqiang Bai, Yeting Li, Wei Ma, Xiaofei Xie, Yuekang Li, Limin Sun, and Yang Liu. 2024. How effective are they? Exploring large language model based fuzz driver generation. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '24)*, 1223–1235. doi:10.1145/3650212.3680355
- [58] Quanjin Zhang, Ye Shang, Chunrong Fang, Siqi Gu, Jianyi Zhou, and Zhenyu Chen. 2024. TestBench: Evaluating class-level test case generation capability of large language models. arXiv:2409.17561 [cs.SE] doi:10.48550/arXiv.2409.17561
- [59] Zipkin. 2024. Zipkin project. <https://github.com/openzipkin/zipkin>. Accessed: 2025-07-18.